

Stream Übungen

Steam und Game

Nutze diese Klassen für folgende Aufgaben.

```
class Steam {  
  
    private final Collection<Game> library = new ArrayList<>();  
  
    public Steam() {  
        library.addAll(List.of(  
            new Game("Horizon Zero Dawn", 743.1),  
            new Game("Monkey Island 2: LeChuck's Revenge", 367.2),  
            new Game("A Plague Tale: Innocence", 409.3),  
            new Game("XOM 2", 664.4),  
            new Game("Palworld", 1000.0), // most popular  
            new Game("Return to Monkey Island", 331.5),  
            new Game("Mario Kart", 782.8),  
            new Game("A Plague Tale: Requiem", 535.0),  
            new Game("A Way Out", 84.6),  
            new Game("The Secret of Monkey Island", 567.2)  
        ));  
    }  
}
```

```
record Game(String name, double popularity) {  
  
    @Override  
    public String toString() {  
        return name + " (" + popularity + ')';  
    }  
}
```

Implementiere die nachfolgenden Aufgaben in der Klasse **Steam**.

findGamesFilteredByMinPopularity

```
/**  
 * @return a new collection of games whose popularity is greater than the given minPopularity (with streams only).  
 */  
public Collection<Game> findGamesFilteredByMinPopularity(double minPopularity) {  
    return null; // TODO  
}
```

findGamesFilteredByMinPopularity: Lösung

```
/**  
 * @return a new collection of games whose popularity is greater than the given minPopularity (with streams only).  
 */  
public Collection<Game> findGamesFilteredByMinPopularity(double minPopularity) {  
    return library.stream()  
        .filter(game -> game.popularity() > minPopularity)  
        .toList();  
}
```

findGamesFilteredByMinAvgPopularity

```
/**  
 * @return a new collection of games whose popularity is greater than the average popularity of all games (with streams only).  
 * Note: Remember DRY.  
 */  
public Collection<Game> findGamesFilteredByMinAvgPopularity() {  
    return null; // TODO  
}
```

findGamesFilteredByMinAvgPopularity: Lösung

```
/**  
 * @return a new collection of games whose popularity is greater than the average popularity of all games (with streams only).  
 * Note: Remember DRY.  
 */  
public Collection<Game> findGamesFilteredByMinAvgPopularity() {  
    var minAvgPopularity = library.stream()  
        .mapToDouble(Game::popularity)  
        .average()  
        .orElse(0.0);  
    return findGamesFilteredByMinPopularity(minAvgPopularity);  
}
```

findNamesSortedByPopularityDescending

```
/**  
 * @return a new collection of game names that contain the search term (with streams only).  
 * The search is case-insensitive.  
 * The names are sorted in descending order of popularity,  
 * i.e., the most popular game (highest value) is at the beginning of the collection.  
 */  
public Collection<String> findNamesSortedByPopularityDescending(String searchTerm) {  
    return null; // TODO  
}
```

findNamesSortedByPopularityDescending: Lösung

```
/**  
 * @return a new collection of game names that contain the search term (with streams only).  
 * The search is case-insensitive.  
 * The names are sorted in descending order of popularity,  
 * i.e., the most popular game (highest value) is at the beginning of the collection.  
 */  
public Collection<String> findNamesSortedByPopularityDescending(String searchTerm) {  
    return library.stream()  
        .filter(game -> game.name().toLowerCase().contains(searchTerm.toLowerCase()))  
        .sorted((game1, game2) -> Double.compare(game2.popularity(), game1.popularity()))  
        .map(Game::name)  
        .toList();  
}
```


findAllGamesWithAcronymizedNames

```
/**
 * @return a new collection of all games with acronymized names.
 * Converts the name of each game into an acronym by keeping only the first letter (or digit) of each word.
 * Example:
 * [Game("The Secret of Monkey Island", 567.2), Game("XOM 2", 664.4)] is converted into:
 * [Game("TSoMI", 567.2), Game("X2", 664.4)]
 * Note: You may solve this method without streams.
 */
public Collection<Game> findAllGamesWithAcronymizedNames() {
    return null; // TODO
}
```

findAllGamesWithAcronymizedNames: Lösung

```
/**
 * @return a new collection of all games with acronymized names.
 * Converts the name of each game into an acronym by keeping only the first letter (or digit) of each word.
 * Example:
 * [Game("The Secret of Monkey Island", 567.2), Game("XOM 2", 664.4)] is converted into:
 * [Game("TSoMI", 567.2), Game("X2", 664.4)]
 * Note: You may solve this method without streams.
 */
public Collection<Game> findAllGamesWithAcronymizedNames() {
    return library.stream()
        .map(game -> {
            var words = game.name().split(" ");
            StringBuilder acronymizedName = new StringBuilder();
            for (String word : words) {
                char firstLetter = word.charAt(0);
                acronymizedName.append(firstLetter);
            }
            return new Game(acronymizedName.toString(), game.popularity());
        })
        .toList();
}
```

threeGamesOfTheDay

```
/**
 * @return a new collection containing three random, different games.
 * Note: You may solve this method without streams.
 * @see java.util.Collections#shuffle
 */
public Collection<Game> threeGamesOfTheDay() {
    return null; // TODO
}
```

threeGamesOfTheDay: Lösung

```
/**  
 * @return a new collection containing three random, different games.  
 * Note: You may solve this method without streams.  
 * @see java.util.Collections#shuffle  
 */  
public Collection<Game> threeGamesOfTheDay() {  
    var randomGames = new ArrayList<>(library);  
    Collections.shuffle(randomGames);  
    return randomGames.subList(0, 3);  
}
```